Count-Min Sketch

Frequency Estimation in Large Data Streams

Daksh Shaurya Swapnil Tridham Akshay
Shinjan G
Indian Statistical Institute
Probability 2 (ISI)

Abstract

How often has a particular item appeared in a data stream so huge that we cannot afford to remember every item? This report answers that question with the Count-Min Sketch. Everything is explained through one running example: a stream of 20 arrivals of 7 different items $A, \ldots, G$ and a small grid of 3×11 counters. We first work the example completely by hand, then express the algorithm in general notation, prove why it works using elementary probability, and finally look at how the size of the stream affects it, where it is used, and where it fails. The technical details of hash functions are deliberately left to the end.

Contents

1 The Problem **3**

1.1 Why we cannot just store the stream 3

1.2 Exact counting and its memory problem 3

1.3 Trading exactness for memory 4

2 The Idea: a Small Grid of Counters **4**

3 A Complete Worked Example **4**

3.1 The stream and the grid 4

3.2 Processing the stream 5

3.3 Query 1: a frequent item, A ($f(A) = 8$) 5

3.4 Query 2: a rare item, G ($f(G) = 1$) 6

3.5 All seven items 6

4 The Algorithm in General Notation **6**

5 Why It Works: the Probability Behind the Sketch **7**

5.1 The extra count in one row 7

5.2 The assumptions behind the guarantee 7

5.3 Expected extra count 8

5.4 The point-query guarantee 8
5.5 Choosing the width and the depth 9
5.6 Checking the theorem on the example 9
5.7 Memory bound and remarks 9

6 How the Stream Affects the Sketch **10**
6.1 Length of the stream 10
6.2 Streams where almost every item is different 10
6.3 Number of distinct items 10
6.4 Combining sketches and forgetting old data 11

7 Where It Is Used **11**
7.1 Heavy hitters 11

8 Hash Functions in Detail **12**

9 Limitations and Conclusion **12**
9.1 Advantages and limitations 12
9.2 Conclusion 13

Central question. How can we estimate the frequency of an item in a massive data stream while using substantially less memory than exact counting?

1 The Problem

A *data stream* is an ordered sequence of observations or events that arrive over time: $x_1, x_2, x_3, \ldots$ Typical examples are network packets, search queries, financial transactions, website clicks, sensor readings and system logs. In this report each observation is called an *item*, and we label items by letters, so a stream looks like

$$A,\ B,\ A,\ C,\ A,\ B,\ D,\ A,\ \ldots$$

The question we care about is: *how many times has a particular item appeared so far?* For an item x this number is its *frequency*,

$$f(x) = \#\{\, i : x_i = x \,\}.$$

In the prefix above, $f(A) = 4$. We write $N = \sum_y f(y)$ for the total number of arrivals so far.

1.1 Why we cannot just store the stream

A small static dataset can be stored completely, but a massive stream often cannot:

- **Too large:** billions or trillions of records.
- **Continuous:** new observations may never stop, so memory cannot grow indefinitely.
- **High velocity:** data arrives too quickly to store first, and each item may have to be processed and discarded immediately (*one-pass processing*).

The number of distinct items may also be enormous, and storing the complete stream is often unnecessary, since usually only summary information is needed. Streaming algorithms therefore summarize the stream using limited memory, processing data as it arrives and without keeping the historical stream.

1.2 Exact counting and its memory problem

The obvious solution is an exact table with one *counter* (a memory cell holding a number) per distinct item, for instance

Item	A	B	C	D	E
Frequency	5	3	2	2	1

(This small table is a separate illustration, unrelated to the 20-arrival stream of Section 3.) Every time an item arrives, its counter goes up by one. The catch is that we need a separate counter for every distinct item:

$$\text{more distinct items} \Longrightarrow \text{more counters} \Longrightarrow \text{more memory}.$$

With 10^9 distinct identifiers, an exact table needs about 10^9 counters, even if we only ever ask about a handful of items.

1.3 Trading exactness for memory

Instead of one counter per item, we keep a small, fixed summary of the stream. The question changes from "how can every frequency be stored exactly?" to "how can frequencies be *estimated* using limited memory?" This is a trade-off:

$$\text{less memory} \iff \text{approximation.}$$

An approximation is only useful if its error can be controlled, and that is where probability enters the story (Section 5).

2 The Idea: a Small Grid of Counters

Here is the whole idea in plain words. Instead of one counter per item, keep a small grid of counters with a few rows and many fewer columns than there are items. In our running example the grid has 3 rows and 11 columns, so only 33 counters, whatever the number of items. (Here "whatever the number of items" refers to the *number* of counters; each counter must still be wide enough in bits to hold its count, see Sections 4 and 6.3.)

Every row owns a fixed *assignment rule* that sends each item to exactly one of the 11 columns of that row, and different rows use different rules. (How such rules are built in practice is the subject of Section 8; for now think of each rule as a fixed lookup that never changes.) Two different items can be sent to the same column in a row; we then say they *collide* in that row, and the counter of that column mixes their counts.

The grid supports two operations:

- **Update** (an item arrives). In every row, find the column the item is sent to and add 1 to that counter. The item itself is not stored.
- **Query** (how often did this item appear?). In every row, read the counter of the column the item is sent to. Report the *smallest* of these numbers as the estimate $\hat{f}$.

Why the name? *Count*: each incoming item increments counters. *Min*: a query takes the minimum of the counters it reads. *Sketch*: a photograph preserves enormous detail, whereas a sketch keeps only what is needed to capture the main structure. The grid does not keep the stream, nor an exact entry for every item; it keeps just enough aggregate information to estimate frequencies. Compact representation with controlled approximation is the central philosophy.

3 A Complete Worked Example

3.1 The stream and the grid

Our stream has $N = 20$ arrivals of 7 distinct items:

$$A, B, A, C, A, B, A, D, A, C, E, A, B, A, F, D, B, C, A, G.$$

For checking our work only (the grid never sees this), the true frequencies of A, B, C, D, E, F, G are $8, 4, 3, 2, 1, 1, 1$, which indeed add up to $N = 20$.

Our grid has 3 rows and 11 columns, and every counter starts at 0. The three assignment rules are fixed before the first arrival and never change:

Item	A	B	C	D	E	F	G
True frequency	8	4	3	2	1	1	1
Row 1 sends it to column	1	5	7	10	4	3	7
Row 2 sends it to column	3	8	7	3	1	8	8
Row 3 sends it to column	9	11	4	10	9	7	4

For example, A is sent to columns $1, 3, 9$ in rows $1, 2, 3$: whenever A arrives, we add 1 to the counters in row 1 column 1, row 2 column 3 and row 3 column 9.

3.2 Processing the stream

Arrival 1 is A. Three counters change from 0 to 1.

Grid after 1 arrival

	1	2	3	4	5	6	7	8	9	10	11
Row 1	1	0	0	0	0	0	0	0	0	0	0
Row 2	0	0	1	0	0	0	0	0	0	0	0
Row 3	0	0	0	0	0	0	0	0	1	0	0

Arrival 2 is B, which goes to columns $5, 8, 11$; arrival 3 is A again, raising its three counters to 2; arrival 4 is C, which goes to columns $7, 7, 4$. After four arrivals the grid is:

Grid after 4 arrivals

	1	2	3	4	5	6	7	8	9	10	11
Row 1	2	0	0	0	1	0	1	0	0	0	0
Row 2	0	0	2	0	0	0	1	1	0	0	0
Row 3	0	0	0	1	0	0	0	0	2	0	1

The remaining 16 arrivals $A, B, A, D, A, C, E, A, B, A, F, D, B, C, A, G$ are processed with exactly the same rule. Collisions begin to appear: for instance, when D arrives it lands on the same column as A in row 2 (column 3), and G lands on the same column as C in rows 1 and 3. The final grid is:

Final grid after all 20 arrivals

	1	2	3	4	5	6	7	8	9	10	11	Row sum
Row 1	8	0	1	1	4	0	4	0	0	2	0	20
Row 2	1	0	10	0	0	0	3	6	0	0	0	20
Row 3	0	0	0	4	0	0	1	0	9	2	4	20

Two facts are worth noticing. Every arrival adds exactly 1 to every row, so each row sums to the number of arrivals processed, 20. And because each update is just "add 1" and addition does not care about order, the final grid is the same for any reordering of the 20 arrivals. Instead of storing every item, we only keep track of how many times each counter was updated.

3.3 Query 1: a frequent item, A ($f(A) = 8$)

A is sent to columns $1, 3, 9$:

Row	counter read	who else shares this column	extra count
1	8	nobody	0
2	10	D (2 arrivals)	2
3	9	E (1 arrival)	1

The estimate is the smallest counter, $\hat{f}(A) = \min\{8, 10, 9\} = 8$, which is exactly the true frequency.

Why the minimum works: every arrival of A added 1 to each of the three counters, so each counter holds the true count *plus* an extra count contributed by the other items sharing its column. Extra counts can only add, never subtract. So every counter is at least the true count, and the smallest counter is the one least spoiled by sharing. Here row 1 is not shared at all, which is why the answer is exact. In general, it is enough that *some* row is only lightly shared to get a good estimate.

3.4 Query 2: a rare item, G ($f(G) = 1$)

G is sent to columns $7, 8, 4$:

Row	counter read	who else shares this column	extra count
1	4	C (3)	3
2	6	B (4) and F (1)	5
3	4	C (3)	3

Every row is shared, so $\hat{f}(G) = \min\{4, 6, 4\} = 4$ although the true frequency is 1: an absolute error of $4 - 1 = 3$, or a relative error of $\frac{4-1}{1} \times 100 = 300\%$ (a 300% overestimate). The size of the error depends on what else lands in the column, not on the item's own count. Items with small frequencies can be greatly affected by collisions, while frequent items are less affected.

3.5 All seven items

Item	A	B	C	D	E	F	G
True frequency f	8	4	3	2	1	1	1
Estimate $\hat{f}$	8	4	3	2	1	1	4
Error $\hat{f} - f$	0	0	0	0	0	0	3

The estimate is never below the true frequency, and here it is exact for six of the seven items; only the rare item G is badly overestimated. This is a toy run meant to illustrate the *mechanics* of the sketch, not its memory savings: seven distinct items need only 7 exact counters, fewer than our 33. The memory saving appears only when the number of distinct items is vastly larger than the number of counters (Section 6).

4 The Algorithm in General Notation

Now that the mechanics are clear, we can name the pieces. The sketch is a table of counters

$$C \in \mathbb{N}^{d \times w}, \qquad C[j, k], \quad 1 \le j \le d,\ 1 \le k \le w,$$

where d is the number of rows (the *depth*; $d = 3$ in our example), w is the number of columns per row (the *width*; $w = 11$ in our example) and h_j is the assignment rule of row j, so $h_j(x) \in \{1, \ldots, w\}$ is the column that row j sends item x to. The table has dw counters, a number fixed once the sketch is built, and initially $C[j, k] = 0$ for all j, k.

Update. When x arrives,

$$C[j, h_j(x)] \leftarrow C[j, h_j(x)] + 1, \qquad j = 1, \ldots, d.$$

One arrival causes exactly d increments, and x itself is not stored.

Query.

$$\hat{f}(x) = \min_{1 \le j \le d} C[j, h_j(x)].$$

In the example, $\hat{f}(A) = \min\{C[1,1], C[2,3], C[3,9]\} = \min\{8, 10, 9\} = 8$.

Why the minimum works, in symbols. Each relevant counter equals the true count plus an extra count $X_j \ge 0$ coming from other items in the same column:

$$C[j, h_j(x)] = f(x) + X_j, \qquad \hat{f}(x) = f(x) + \min_j X_j.$$

In the example, $X = (0, 2, 1)$ for A and $X = (3, 5, 3)$ for G. Since $X_j \ge 0$ we get $\hat{f}(x) \ge f(x)$ always: for non-negative updates the sketch never underestimates, and its only error is the extra mass caused by collisions.

Cost. An update touches one counter per row, so it takes $O(d)$ time; a query reads one counter per row, so it also takes $O(d)$ time. The memory is $O(dw)$ *counters*. In Section 5 we choose $w = O(1/\varepsilon)$ and $d = O(\ln(1/\delta))$, so the number of counters, $O\big(\frac{1}{\varepsilon} \ln \frac{1}{\delta}\big)$, does not depend on the number of distinct items or on the length N of the stream. The *number of bits* per counter is a separate matter: a counter must be large enough to hold the largest value it will ever reach, which is at most N, so about $\lceil \log_2(N+1) \rceil$ bits per counter suffice, and total memory in bits is $O\big(\frac{1}{\varepsilon} \ln \frac{1}{\delta} \cdot \log N\big)$.

5 Why It Works: the Probability Behind the Sketch

5.1 The extra count in one row

Fix an item x and a row j. The extra count X_j is the total frequency of the *other* items sharing x's column in row j:

$$X_j = C[j, h_j(x)] - f(x) = \sum_{y \ne x} f(y)\, \mathbf{1}\{h_j(y) = h_j(x)\} \ \ge 0,$$

where $\mathbf{1}\{h_j(y) = h_j(x)\}$ equals 1 if item y is sent to the same column as x in row j and 0 otherwise. In the example, for A in row 2 the only other item in its column is D, so $X_2 = f(D) = 2$, matching the table above.

5.2 The assumptions behind the guarantee

From now on the assignment rules are chosen *at random* before the stream starts. We assume:

1. **Fixed stream.** The frequencies $f(y) \ge 0$ and $N = \sum_y f(y)$ are fixed before the rules are drawn.
2. **Random rules.** For two different items $x \ne y$ and any row j, $\mathbb{P}\big(h_j(x) = h_j(y)\big) = \frac{1}{w}$ (the *pairwise collision property*), and the rules $h_1, \ldots, h_d$ of different rows are drawn *independently*.
3. **Non-negative updates.** Every update adds $+1$.

All randomness lies in the rules; the stream is not random. These two properties of the rules, the pairwise collision property and independence across rows, are exactly what the analysis requires. A hash function whose outputs merely *look* uniform is not enough by itself to establish the theorem. Why these assumptions are reasonable is discussed in Section 8.

5.3 Expected extra count

By linearity of expectation and the pairwise collision property,

$$\mathbb{E}[X_j] = \sum_{y \neq x} f(y)\,\mathbb{P}\big(h_j(y) = h_j(x)\big) = \frac{1}{w}\sum_{y \neq x} f(y) = \frac{N - f(x)}{w} \leq \frac{N}{w}.$$

Only the pairwise property is needed; independence within a row is not required. On average, a row adds at most a $\frac{1}{w}$-share of the whole stream to the count of x.

In the example ($N = 20$, $w = 11$): $\mathbb{E}[X_j] = \frac{20-8}{11} \approx 1.09$ for A and $\frac{20-1}{11} \approx 1.73$ for G. Our single draw gave extra counts $0, 2, 1$ for A (average 1) and $3, 5, 3$ for G (average ≈ 3.67). An expectation is an average over random draws, so one draw can land above or below it.

5.4 The point-query guarantee

Theorem 1 (Count-Min Sketch error bound)**.** *Let C be a Count-Min Sketch with d independent assignment rules $h_1, \ldots, h_d$, each sending items to w columns. For any* fixed *item x with true count $f(x)$, the estimate $\hat{f}(x)$ satisfies*

$$f(x) \leq \hat{f}(x) \leq f(x) + \varepsilon N \quad \textit{with probability at least } 1 - \delta,$$

provided

$$w = \left\lceil \frac{e}{\varepsilon} \right\rceil \quad \textit{and} \quad d = \left\lceil \ln \frac{1}{\delta} \right\rceil,$$

where $N = \sum_y f(y)$ is the total number of items in the stream, ε is the error tolerance and δ is the failure probability.

Proof. Lower bound. Every counter $C[j, h_j(x)]$ is incremented each time x appears and other items can only add to it, so $C[j, h_j(x)] = f(x) + X_j \geq f(x)$ for every row j. Hence $\hat{f}(x) = \min_j C[j, h_j(x)] \geq f(x)$ with probability 1.

Upper bound, one row. We use Markov's inequality: if $X \geq 0$ and $t > 0$ then $\mathbb{P}(X \geq t) \leq \mathbb{E}[X]/t$. Apply it to $X_j \geq 0$ with threshold $t = \varepsilon N$:

$$\mathbb{P}(X_j > \varepsilon N) \leq \mathbb{P}(X_j \geq \varepsilon N) \leq \frac{\mathbb{E}[X_j]}{\varepsilon N} \leq \frac{N/w}{\varepsilon N} = \frac{1}{\varepsilon w} \leq \frac{1}{e},$$

because $w = \lceil e/\varepsilon \rceil \geq e/\varepsilon$. So each row is "bad" (its extra count exceeds εN) with probability at most $1/e$.

Upper bound, all rows. The minimum exceeds $f(x) + \varepsilon N$ only if *every* row is bad:

$$\{\hat{f}(x) > f(x) + \varepsilon N\} = \bigcap_{j=1}^{d} \{X_j > \varepsilon N\}.$$

Each X_j depends only on h_j, and the rules are independent, so the probabilities multiply:

$$\mathbb{P}\big(\hat{f}(x) > f(x) + \varepsilon N\big) = \prod_{j=1}^{d} \mathbb{P}(X_j > \varepsilon N) \leq \left(\frac{1}{e}\right)^d = e^{-d} \leq \delta,$$

since $d = \lceil \ln \frac{1}{\delta} \rceil \geq \ln \frac{1}{\delta}$. Combining with the lower bound, $\mathbb{P}\big(f(x) \leq \hat{f}(x) \leq f(x) + \varepsilon N\big) \geq 1 - \delta$. □

5.5 Choosing the width and the depth

- **Width controls accuracy.** One row is bad with probability at most $\frac{1}{\varepsilon w}$; we need this to be at most $\frac{1}{e}$, i.e. $w \geq e/\varepsilon$. A smaller ε needs a larger w.
- **Depth controls confidence.** d independent rows are all bad with probability at most e^{-d}; we need $e^{-d} \leq \delta$, i.e. $d \geq \ln \frac{1}{\delta}$. A smaller δ needs a larger d.

Why the number e? Let $q = \frac{1}{\varepsilon w}$ be the per-row failure bound. Then $w = \frac{1}{\varepsilon q}$ and $d = \frac{\ln(1/\delta)}{\ln(1/q)}$ (ignoring ceilings), so

$$dw = \frac{\ln(1/\delta)}{\varepsilon} \cdot \frac{1}{q \ln(1/q)}.$$

The product $q \ln(1/q)$ is largest at $q = 1/e$, which therefore uses the fewest counters. At $q = 1/e$ we get $w = \frac{e}{\varepsilon}$, $d = \ln \frac{1}{\delta}$ and $dw = \frac{e \ln(1/\delta)}{\varepsilon}$, exactly our choice.

5.6 Checking the theorem on the example

We chose the grid size for $\varepsilon = 0.25$ and $\delta = 0.05$:

$$w = \left\lceil \frac{e}{0.25} \right\rceil = \lceil 10.87 \rceil = 11, \qquad d = \lceil \ln 20 \rceil = \lceil 2.996 \rceil = 3.$$

With $N = 20$ the error bound is $\varepsilon N = 0.25 \cdot 20 = 5$, so for a fixed item, with probability at least $1 - \delta = 0.95$ (over the random rules) the theorem guarantees $f(x) \leq \hat{f}(x) \leq f(x) + 5$; the lower bound always holds. Both queries are inside the bound (the errors are 0 for A and 3 for G), but because the guarantee is a fraction of N, an error of 5 can be large compared with a rare item's true frequency.

For A we know $\mathbb{E}[X_j] \approx 1.09$, so Markov gives $\mathbb{P}(X_j \geq 5) \leq \frac{1.09}{5} \approx 0.22$. Without knowing $f(A)$ we use $\mathbb{E}[X_j] \leq N/w$ instead:

$$\mathbb{P}(X_j > \varepsilon N) \leq \frac{1}{\varepsilon w} = \frac{1}{0.25 \cdot 11} \approx 0.364 \leq \frac{1}{e} \approx 0.368.$$

With $d = 3$ independent rows the general theorem gives

$$\mathbb{P}(\text{all three rows bad}) \leq \left(\frac{1}{e}\right)^3 \approx 0.0498 \leq \delta = 0.05,$$

while for this particular item A the sharper figure is $0.2183 \approx 0.0104$.

The probability is over the random choice of rules, made before the stream; the grid above is *one draw*, and the bound says nothing about one fixed table. Markov's inequality is loose: the realised extra counts for A are at most 2, and real failure probabilities are usually far below e^{-d}.

5.7 Memory bound and remarks

Using $w = O(\frac{1}{\varepsilon})$ and $d = O(\ln \frac{1}{\delta})$,

$$dw = O\left(\frac{1}{\varepsilon} \ln \frac{1}{\delta}\right):$$

the *number of counters* depends on the desired accuracy and confidence, not on the number of distinct items and not on the stream length N. The number of bits needed *per counter* does depend on the largest count it must hold (at most N, i.e. about $\log_2 N$ bits), so total memory in bits grows only logarithmically with N. Two remarks on the guarantee:

- **One item at a time.** δ is the failure probability of *one fixed* query. For all m items at once use a union bound: $d = \lceil \ln(m/\delta) \rceil$.
- **Oblivious stream.** The stream must not depend on the random rules.

6 How the Stream Affects the Sketch

6.1 Length of the stream

The guarantee $f(x) \le \hat{f}(x) \le f(x) + \varepsilon N$ means that, for a fixed sketch, a longer stream gives a larger error bound. Take a realistic sketch with $\varepsilon = 0.001$ and $\delta = 0.01$, so $w = \lceil e/0.001 \rceil = 2719$ and $d = \lceil \ln 100 \rceil = 5$:

N	error bound εN	average column load N/w
10^6	10^3	≈ 368
10^9	10^6	$\approx 3.7 \times 10^5$

To keep a fixed error E we need $\varepsilon N \le E$, so $w = \lceil eN/E \rceil$; for example $E = 100$ and $N = 10^6$ give $w = 27{,}183$.

6.2 Streams where almost every item is different

Suppose almost every item occurs once: $f(x) \approx 1$ and the number of distinct items $m \approx N$. If the N items spread roughly evenly, then each column holds about N/w counts, so estimates will often be of the order of N/w, whether $f(x) = 1$ or 0. This is an intuition under roughly balanced bucket loads, not a guarantee for every item: actual bucket loads vary, and the minimum over the rows can differ from N/w.

As an illustration, with $\varepsilon = 0.001$, $w = 2719$ and $N = 10^9$, a typical estimate is of the order of 3.7×10^5. The theorem still holds, but εN is useful only when $f(x) \gg \varepsilon N$. Here the sketch cannot reliably tell an item seen once from an item never seen; for rare-item frequencies, an exact table is more appropriate.

6.3 Number of distinct items

The sketch above has $dw = 5 \cdot 2719 = 13{,}595$ counters, about 54 KB with 4-byte counters, whatever the number of distinct items m is (the 4-byte counter width itself is an assumption: it suffices only while no counter exceeds $2^{32} - 1$, so very long streams would need wider counters):

distinct items m	exact table (4-byte counters)	Count-Min
10^4	40 KB	54 KB
10^6	4 MB	54 KB
10^9	4 GB	54 KB

Only counter storage is counted; the key and dictionary overhead of the exact table is ignored. The sketch is smaller only when $m > dw = 13{,}595$. For few distinct items an exact table is smaller and exact; the saving is real only when $m \gg dw$.

6.4 Combining sketches and forgetting old data

- **Sketches merge.** With the same d, w and assignment rules, the grids of two streams S_1, S_2 satisfy $C^{(S_1 \circ S_2)} = C^{(S_1)} + C^{(S_2)}$ entry by entry, and the guarantee holds with $N = N_1 + N_2$. Sketches built on separate machines can therefore be combined by adding them.
- **Old data stays.** Counters never forget, so $\hat{f}(x)$ never decreases. For drifting streams use time windows or periodic resets.

7 Where It Is Used

In each application below the sketch estimates the frequency of *tracked* items; it stores no item names, so a separate candidate-generation method is needed to discover which items are heavy or popular.

- **Network traffic monitoring.** A network produces a never-ending stream of packets. Here x is an IP address and $f(x)$ the number of packets from it; we can estimate it without a counter for every address on the internet.
- **Web and search analysis.** x is a search query such as "weather" and $\hat{f}(x)$ estimates how often it was searched over the monitored stream or window. Frequent queries are exactly where the sketch is accurate.
- **Social media.** x is a hashtag such as #AI; with a time window or periodic reset, the sketch helps track frequently occurring hashtags, and it stays the same small size however many posts arrive.
- **Logs and databases.** x is an API endpoint; we get useful usage statistics without storing the whole log history.

7.1 Heavy hitters

A *heavy hitter* is an item that appears very often, for example $f(x) \geq 0.3N$ (at least 30% of the stream). The sketch only answers "how many times is x?", so we keep a short list of candidates: when an item arrives, update the grid, compute $\hat{f}(x)$, and if $\hat{f}(x) \geq 0.3N$ keep x in the list. The candidates come from the arriving items themselves (each is checked on arrival) or from another discovery method; only the short list stores item names. Since N grows as the stream proceeds, items kept early should be re-checked against the final N.

Tiny example. Take the stream $A, A, A, B, C, A, B, D, A, A$. Then $N = 10$ and the cut-off is $0.3N = 3$. The frequencies are $f(A) = 6$, $f(B) = 2$, $f(C) = 1$, $f(D) = 1$, so only A is a heavy hitter.

Why we never miss one. Since $\hat{f}(x) \geq f(x)$ always, a truly heavy item always passes the test $\hat{f}(x) \geq 0.3N$, so no heavy hitter is missed (this part is deterministic, for non-negative updates). The converse needs care. A light item x is wrongly kept only if its overcount lifts $\hat{f}(x)$ to at least $0.3N$. For one *fixed* item, the overcount is at most εN with probability at least $1 - \delta$, so with that probability an item with $f(x) < (0.3 - \varepsilon)N$ is correctly rejected. Two qualifications apply:

- The guarantee is per fixed item. To have all m candidate items classified correctly *simultaneously*, use the union bound of Section 5.7 (take $d = \lceil \ln(m/\delta) \rceil$); the failure probability δ of a single query does not carry over to all of them at once.

- An item whose true frequency lies just below the cut-off, with $(0.3-\varepsilon)N \le f(x) < 0.3N$, can be falsely included even when the sketch's error is within εN. A small ε narrows this borderline band but never removes it.

8 Hash Functions in Detail

Throughout the report each row used an "assignment rule". The standard name is a *hash function.*

Definition 1 (Hash function)**.** *A function that takes an item as input and maps it to a fixed range of bucket positions (here, the columns of a row), so that the item can be assigned to a specific counter in the sketch.*

A compact summary must squeeze arbitrary items into a small number of counters:

$$\text{item} \longrightarrow \text{hash function} \longrightarrow \text{bucket}, \qquad h(x) \in \{1, \ldots, w\}.$$

For the analysis we do not need to know how a hash function is implemented internally; we treat it as a black box $x \to h \to j$. What matters are its *statistical properties*, which determine the quality of the estimate. A black-box description alone does not supply these properties: the theorem of Section 5.4 holds only if the hash functions actually have the pairwise collision property and are independent across rows, and outputs that merely look uniform are not enough to establish it.

1. **Uniformity.** A desirable hash function spreads items evenly over the w buckets, ideally $\mathbb{P}(h(x) = j) = \frac{1}{w}$ for every bucket j. The intuition is that no bucket is systematically preferred, the counters receive a balanced load, and excessive collisions become less likely. This is an intuition about uniform hashing, separate from the pairwise collision property; the proof itself needs only the pairwise collision property.
2. **Low collision probability.** Collisions are unavoidable when the number of possible items is much larger than the number of buckets. The goal is not to eliminate collisions but to control their effect, and the proof requires exactly the pairwise collision property $\mathbb{P}(h_j(x) = h_j(y)) = \frac{1}{w}$ stated in Section 5.2.
3. **Independence across rows.** If x collides badly with others in one row, a different row uses a different hash function and may have a completely different collision pattern. Independent rows make it unlikely that collisions are bad in every row, and this independence is what allows the failure probabilities to multiply in the proof.

9 Limitations and Conclusion

9.1 Advantages and limitations

Advantages	**Limitations**
Memory efficient	Approximate; collisions only overestimate
Fast updates and queries	Error εN is absolute: rare items are not reliable
Single-pass processing	Near-uniform streams: useless
Suitable for continuous streams	Saves memory only if distinct items $\gg dw$
Scalable	Cannot list items without a candidate set
Controlled probabilistic error	Needs good hashes, an oblivious stream and non-negative updates

9.2 Conclusion

Exact counting of a massive stream is expensive because the number of distinct items can be enormous. The Count-Min Sketch replaces the exact table by a small grid of counters: each arriving item adds 1 to one counter per row, and a query returns the smallest of the item's counters. Collisions can only add extra counts, so the estimate never falls below the truth, and independent rows make it unlikely that every row is badly spoiled. The price and the payoff are captured by one pair of formulas,

$$w = \left\lceil \frac{e}{\varepsilon} \right\rceil, \quad d = \left\lceil \ln \frac{1}{\delta} \right\rceil \qquad \text{so that the number of counters} = O\left(\frac{1}{\varepsilon} \ln \frac{1}{\delta}\right).$$

Final takeaway. The Count-Min Sketch does not try to remember the entire stream. It remembers just enough to answer frequency queries quickly, using very little memory, with a mathematically controlled error. The catch is that the error is measured against N: estimates are reliable for frequent items and uninformative for rare ones.

Use of AI

AI (Claude, by Anthropic) was used to make the original presentation (slides and LaTeX source) and to run sanity checks on the worked example, calculations and formulas.

References

[1] Video lectures on the Count-Min Sketch (YouTube), accessed 2026:
`https://www.youtube.com/playlist?list=PL2mpR0RYFQsAR5RyB54FyEE9vUiGtCSZM`,
`https://www.youtube.com/playlist?list=PLM_KIlUOWoXmkV4QBa1Dg8PtJaHTdWHwRS`,
`https://www.youtube.com/watch?v=ibxXO-b14j4`.

[2] T. Roughgarden and G. Valiant. Lecture #2: Approximate Heavy Hitters and the Count-Min Sketch. CS168: The Modern Algorithmic Toolbox, Stanford University, 2 April 2015.
`https://theory.stanford.edu/~tim/s15/l/l2.pdf`

[3] A. Moitra. Problem Set 2. 6.854J/18.415J Advanced Algorithms, Massachusetts Institute of Technology, 2016.
`https://people.csail.mit.edu/moitra/docs/6854hw2.pdf`

[4] B. Sigurleifsson, A. Anbarasu and K. Kangur. An Overview of Count-Min Sketch and its Applications. EasyChair Preprint No. 879, 2019.
`https://easychair.org/publications/preprint/gNlw/open`

[5] N. Limaye. Lecture 7: Count-Min and Count sketches. CS602 Applied Algorithms lecture notes, IIT Bombay, 2014.
`https://www.cse.iitb.ac.in/~nutan/courses/cs602-13/notes/lec7.pdf`

[6] Scribe notes 11. COMP 480/580, Rice University, Fall 2024.
`https://www.cs.rice.edu/~as143/COMP480_580_Fall24/scribe/scribe11.pdf`